explain by taking this packet as an example - dst - ff:ff:ff:ff:ff:ff, src - 00:00:00:00:00:01, ether type -0x88ba

Great example. Let's take your **exact Ethernet packet** and walk through **how it goes through the AI-based DPI pipeline step by step**.

Your packet:

Destination MAC : ff:ff:ff:ff:ff:ff
Source MAC : 00:00:00:00:00:01
EtherType : 0x88BA

EtherType **0x88BA** corresponds to **Sampled Value (SV)** traffic used in IEC-61850 substation networks.

Entity reference:
IEC 61850 Sampled Value

Since you previously mentioned working with **SV packets**, this is actually a perfect example.

---

1 Raw Ethernet Packet

Your Ethernet frame begins like this (in bytes):

ff ff ff ff ff ff 00 00 00 00 00 01 88 ba

Breakdown:

Field	Bytes
Destination MAC	ff ff ff ff ff ff
Source MAC	00 00 00 00 00 01
EtherType	88 ba

If we convert to **decimal values**:

[255,255,255,255,255,255,
0,0,0,0,0,1,
136,186]

This is the **first 14 bytes** of the packet.

After this comes the **SV payload**.

Example simplified frame:

ff ff ff ff ff ff
00 00 00 00 00 01
88 ba
01 02 03 04 05 06 ...

---

2 Packet Converted to ML Input

Your model requires **1500 bytes per packet**.

So the packet becomes:

[255,255,255,255,255,255,
0,0,0,0,0,1,
136,186,
payload bytes...,
0,0,0,0,...]

Padding zeros are added if the packet is shorter.

Final vector size:

1500

So your numpy array row looks like:

[255,255,255,255,... ,0]

Dataset format:

(samples, features)

(178,1500)

Meaning:

178 packets
1500 bytes each

---

3 Normalization

Bytes are converted from **0-255** → **0-1**.

Example conversion:

Byte	Normalized
255	1.0
0	0.0
136	0.533
186	0.729

So your packet becomes:

[1.0,1.0,1.0,1.0,1.0,1.0,
0.0,0.0,0.0,0.0,0.0,0.003,
0.533,0.729,
...]

---

4 Reshape for CNN

Original shape:

(1500)

CNN requires:

(1500,1)

So packet becomes:

[
[1.0]
[1.0]
[1.0]
[1.0]
[1.0]
[1.0]
[0.0]

[0.0]
...
]

Meaning:

1500 steps
1 channel

---

5 CNN Begins Processing

Model architecture:

Input (1500,1)
↓
Embedding
↓
Conv1D
↓
Conv1D
↓
Conv1D
↓
Pooling
↓
Dense
↓
Softmax

Now let's see what happens.

---

6 Convolution Detects Patterns

First convolution layer:

Conv1D(64 filters, kernel=5)

Kernel size:

5 bytes

The filter slides across packet bytes.

Example window:

[255,255,255,255,255]

This corresponds to:

broadcast MAC address

Another window:

[136,186,...]

Which represents:

EtherType 0x88BA

The CNN learns that this pattern indicates:

Sampled Value traffic

---

7 Feature Extraction

Filters detect patterns like:

Pattern	Meaning
ff ff ff ff ff ff	broadcast frame
88 ba	SV protocol
specific payload	measurement data

So CNN extracts **protocol fingerprints**.

Example feature vector:

[0.1,0.3,0.9,0.2,...]

Each number indicates **strength of a detected pattern**.

8️⃣ Global Max Pooling

Now the model checks:

Did this pattern appear anywhere in the packet?

Example:

Feature map:

[0.1,0.4,0.8,0.2,0.3]

Pooling result:

0.8

Meaning:

SV protocol pattern detected

9️⃣ Dense Layer Combines Features

After pooling we might have:

256 features

Dense layer:

Dense(128)

Combines protocol signals like:

broadcast frame
SV EtherType
payload structure
packet size

🔟 Final Classification

Final layer:

Dense(num_classes)
Softmax

Example classes:

Class	Protocol
0	HTTP
1	DNS
2	SV
3	YouTube

Output example:

[0.02,0.01,0.95,0.02]

Meaning:

95% probability → SV packet

Prediction:

class = 2

---

11 Running on Hardware

Once the model is trained it is deployed on the **QCS9075**.

Using runtime from:

Qualcomm AI Engine Direct SDK

Execution:

snpe-net-run

Model format:

DLC

Inference runs on:

Hexagon DSP / HTP NPU

So CPU just feeds packet data, while the **AI accelerator performs classification**.

---

12 Real Networking Pipeline

In a real system the flow would be:

```
NIC receives packet
      ↓
Linux kernel
      ↓
packet captured
      ↓
1500-byte tensor
      ↓
AI model
      ↓
traffic classification
```

Example result:

Packet → SV traffic

---

13 Why This Is Powerful

Traditional DPI:

```
if (ethertype == 0x88BA)
   classify as SV
```

AI DPI learns automatically.

It can detect:

encrypted protocols
unknown traffic
anomalies

malware traffic

✅ Since you previously mentioned **Linux kernel networking and SV traffic filtering**, the **most powerful architecture** would be:

NIC
↓
XDP filter
↓
SV packets → custom qdisc
↓
AI model
↓
traffic analytics

This combines **kernel datapath + AI inference**.

If you want, I can also show you something very interesting:

How a CNN filter actually detects the `0x88BA` pattern in the packet byte stream (visual step-by-step). That usually makes the model behavior crystal clear.

yes

Great 👍 Let's **visually see how a CNN filter detects the `0x88BA` EtherType** from your packet.

Your packet header again:

Dst MAC : ff:ff:ff:ff:ff:ff
Src MAC : 00:00:00:00:00:01
EtherType : 0x88BA

Protocol:

IEC 61850 Sampled Value

1️⃣ Raw Packet Bytes

Ethernet header in bytes:

ff ff ff ff ff ff
00 00 00 00 00 01
88 ba

Convert to decimal:

[255,255,255,255,255,255,
0,0,0,0,0,1,
136,186]

Position in packet:

Byte index	Value	Meaning
0–5	255	Broadcast MAC
6–11	0..1	Source MAC
12	136	EtherType byte 1
13	186	EtherType byte 2

---

2 Packet Converted to CNN Input

The packet becomes a **1500-byte vector**.

Example beginning:

[255,255,255,255,255,255,
0,0,0,0,0,1,
136,186,
payload....]

---

3 CNN Sliding Window

First convolution layer:

Conv1D(kernel_size = 5)

So CNN looks at **5 bytes at a time**.

Sliding window moves like this:

Step 1
[255,255,255,255,255]

Step 2
[255,255,255,255,255]

Step 3
[255,255,255,255,0]

Step 4
[255,255,255,0,0]

Step 5
[255,255,0,0,0]

...

Step 10
[0,1,136,186, payload]

---

4 CNN Filter Example

Suppose the CNN learned a filter for **SV EtherType detection**.

Example filter weights:

[0.1 , 0.2 , 0.9 , 1.0 , 0.3]

When sliding window hits:

[0,1,136,186,x]

the filter produces a **large activation value**.

Meaning:

Pattern detected

---

5 Visual Pattern Detection

Think of CNN like this:

```
Packet Byte Stream
---------------------------------------------------
ff ff ff ff ff ff 00 00 00 00 00 01 88 ba payload
                            ↑
                        pattern here
```

CNN filter recognizes:

88 ba

Which corresponds to:

Sampled Value Protocol

---

6 Feature Map Output

After convolution the CNN produces **feature maps**.

Example:

Feature map for SV pattern

[0.01,0.03,0.05,0.08,0.02,0.01,0.02,0.90,0.10,...]

High value:

0.90

means:

SV protocol pattern detected

---

7 Global Max Pooling

Pooling selects the **strongest signal**.

Example:

max([0.01,0.03,0.05,0.08,0.02,0.01,0.02,0.90]) = 0.90

Meaning:

SV signature exists in packet

---

8 Final Prediction

Dense + softmax layer:

Output:

[HTTP:0.02, DNS:0.01, SV:0.96, OTHER:0.01]

Prediction:

SV packet

---

9 Running on IQ9 Hardware

This inference runs on the AI accelerator of

QCS9075

using runtime from

Qualcomm AI Engine Direct SDK.

Execution command example:

```
snpe-net-run \
--container dpi_model.dlc \
--use_dsp
```

The model runs on:

Hexagon DSP / HTP NPU

instead of CPU.

🔟 Real Networking Flow (Your Case)

Based on your **Linux networking experience**, the pipeline could be:

NIC
↓
Linux kernel
↓
XDP / TC hook
↓
Extract packet bytes
↓
AI model inference
↓
Classify traffic (SV / DNS / HTTP)

For example:

ff:ff:ff:ff:ff:ff 00:00:00:00:00:01 88ba
↓
AI model
↓
Detected → SV packet

✅ **Important insight**

For your example packet, the **AI is actually learning multiple things**:

broadcast destination
EtherType pattern
payload structure
packet length

Not just `0x88BA`.

💡 If you want, I can also explain **something very interesting for networking engineers**:

How to run this DPI model at line rate using XDP + NPU on IQ9 (without copying packets to user space).

This is the **architecture used in AI-powered smart NICs and routers**.

explain me this clearly - meaning: we have178 packets, each having 1500 bytes (padded or truncated based on requirement) The tensor might look loke this: (samples, features) (178, 1500) Example normalization: 0 → 0.0 255 → 1.0 now the array becomes, arr = [[0.23, 0.01, 0.87, ..., 0.00], ← packet 1 [0.45, 0.67, 0.00, ..., 0.12], ← packet 2 [0.11, 0.98, 0.76, ..., 0.00], ← packet 3 ...] 2. Now we create labels from the file names (training data): prepare_dataset.py (check this file) here, X = [array(178,1500), array(210,1500), array(150,1500), ...] y = [array(178,), array(210,), array(150,), ...]

Let's go **step-by-step with a very clear example**, because this part is the **foundation of your CNN DPI pipeline**.

You are starting from **PCAP packets** and converting them into **numerical tensors for a CNN model**.

1️⃣ What `(178,1500)` actually means

When you ran:

```
arr = np.load("AIMchat1.pcapng.feather.npy")
print(arr.shape)
```

Output:

```
(178,1500)
```

Meaning:

Dimension	Meaning
178	number of packets
1500	bytes per packet

So your dataset looks like this:

```
178 packets
↓
each packet represented by 1500 bytes
```

So the structure is:

(samples , features)

(178 , 1500)

---

2 What is a sample

In machine learning:

sample = one training example

In your case:

1 packet = 1 sample

So:

178 packets = 178 samples

---

3 What is a feature

A feature is **an input value describing the sample**.

For packet DPI:

each byte = one feature

So:

1500 bytes → 1500 features

---

4 What one row actually looks like

Example packet:

```
ff ff ff ff ff ff
00 00 00 00 00 01
88 ba
aa aa aa ...
```

Converted to decimal:

255 255 255 255 255 255
0 0 0 0 0 1
136 186
170 170 ...

This becomes one row:

[255,255,255,255,255,255,0,0,0,0,0,1,136,186,170,170,...]

Length:

1500 values

---

5 Why packets are padded or truncated

Packets in real networks have **different lengths**:

Example:

Packet	Size
ARP	60 bytes
SV	126 bytes
TCP	1500 bytes

But **CNN needs fixed input size**.

So you force:

all packets = 1500 bytes

Rules:

If packet < 1500

Pad with zeros

Example:

Original packet = 100 bytes

[100 bytes data]

↓

[100 bytes data + 1400 zeros]

If packet > 1500

Cut extra bytes

first 1500 bytes kept

---

6 Why normalization is done

Original byte values:

0 → 255

Neural networks learn better when values are small.

So normalize:

normalized_value = byte / 255

Examples:

Byte	Normalized
0	0.0
128	0.50
255	1.0

Example packet row after normalization:

[1.0,1.0,1.0,1.0,1.0,1.0,
0.0,0.0,0.0,0.0,0.0,0.0039,
0.53,0.73,

0.67,0.67,0.67...]

So your dataset becomes:

arr = [

[0.23,0.01,0.87,...], ← packet 1
[0.45,0.67,0.00,...], ← packet 2
[0.11,0.98,0.76,...], ← packet 3

]

shape = (178,1500)

---

7 Now the next step: creating labels

Machine learning needs:

X → input data
y → correct answer

Example:

Packet	Protocol	Label
AIMchat	chat traffic	0
YouTube	video	1
SV	power grid	2

---

8 How `prepare_dataset.py` organizes data

Suppose you have files:

AIMchat1.pcapng.feather.npy
AIMchat2.pcapng.feather.npy
Youtube1.pcapng.feather.npy

SV1.pcapng.feather.npy

Each file contains packets.

Example:

AIMchat1 → (178,1500)
AIMchat2 → (210,1500)
SV1 → (150,1500)

So:

X = [

array(178,1500), ← AIMchat1 packets
array(210,1500), ← AIMchat2 packets
array(150,1500) ← SV packets

]

---

9 Labels y

For each packet we assign a label.

Example:

AIMchat → label 0
SV → label 1
Youtube → label 2

So label arrays become:

y = [

array(178,) → [0,0,0,0,0,...] AIMchat packets
array(210,) → [0,0,0,0,...] AIMchat packets
array(150,) → [1,1,1,1,...] SV packets

]

Meaning:

each packet gets one label

🔟 Final dataset after merging

The code merges everything.

Example:

X.shape

(33900 , 1500)
y.shape

(33900,)

Meaning:

33900 packets
each packet → 1500 features

1️⃣1️⃣ Next step before CNN

CNN expects:

(sequence_length , channels)

So reshape:

(33900 ,1500)

↓

(33900 ,1500 ,1)

Meaning:

1500 bytes
1 feature channel

1️⃣2️⃣ Complete pipeline

PCAP packets
↓
Extract packet bytes
↓
Pad / truncate → 1500 bytes
↓
Normalize (0-255 → 0-1)
↓
Dataset tensor
(178 ,1500)
↓
Create labels
↓
Train CNN

✅ Since you are building **AI-based DPI for networking**, the **next important concept** is understanding:

Why CNN is used instead of traditional DPI for packet classification.

If you want, I can also explain **how a CNN actually detects protocols like 0x88BA or HTTP inside packet bytes**, which is the **core idea behind AI-based DPI systems.**